



Neural Networks



Contenido

1

Conceptos
Básicos

2

**Tipos de Redes
Neuronales**
FFNN, CNN, RNN, GAN

3

FFNN
MLP y Regularización



Conceptos

¿Que son las Redes Neuronales (ANN)?

“

Las ANN son una **forma de AI** inspirada en la estructura y función del **cerebro humano**. Estas **procesan información** y **toman decisiones** basadas en patrones previamente aprendidos.

”

¿Qué es Deep Learning (DL)?

“

Es una **subcategoría de Machine Learning** que se enfoca en la construcción de modelos con **varias capas de representación** (profundidad) que aprenden directamente de datos.

”

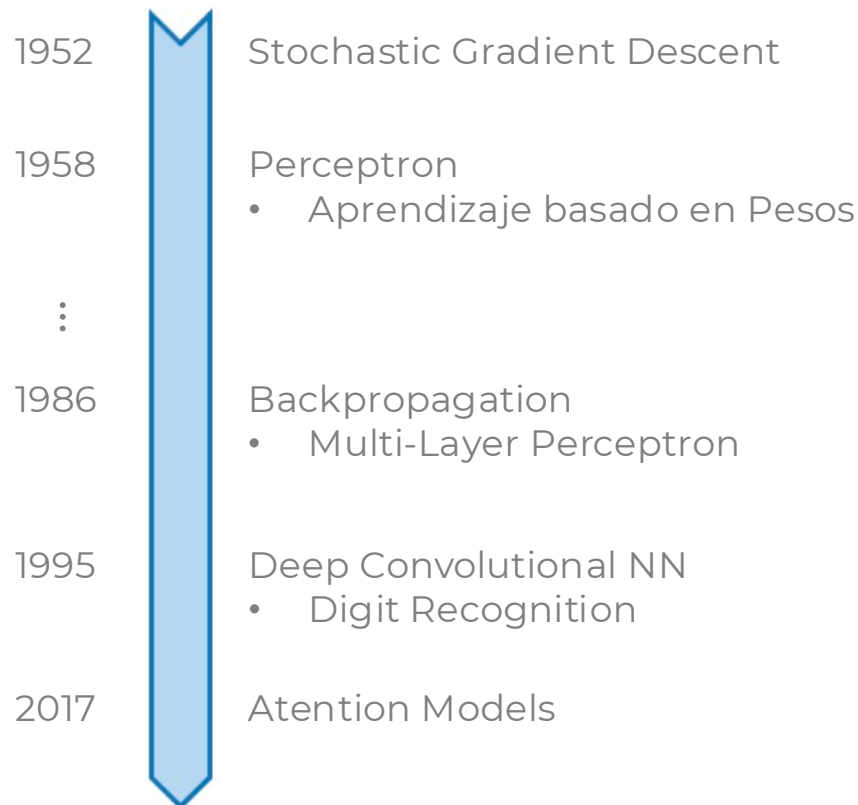
Nota: DL es un tipo de ANN, pero no todo ANN es un tipo de DL: Perceptron, CNN de 1 capa, Adaline, y otros.



Herramientas

Por qué estudiar ahora? Deep Learning

Las ANN tienen décadas de existencia. por qué resurgen?



I. Big Data

- Grandes Datasets
- Facil recolección y almacenamiento



II. Hardware

- GPUs, TPU
- Paralelización Masiva



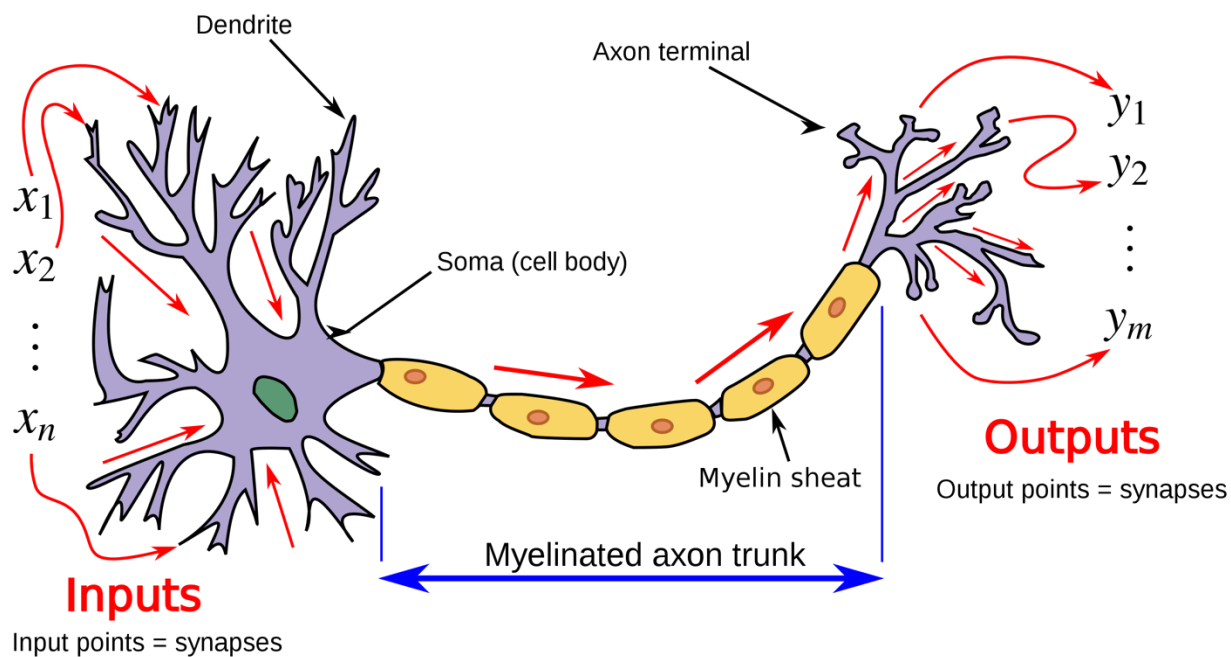
III. Software

- Técnicas Mejoradas
- Nuevos Modelos
- Mejores Librerías
- Open Source

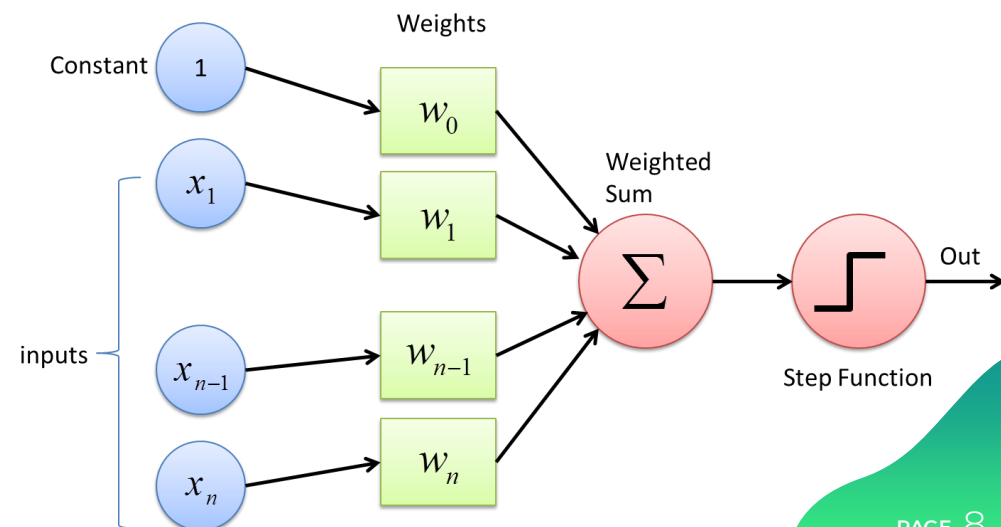
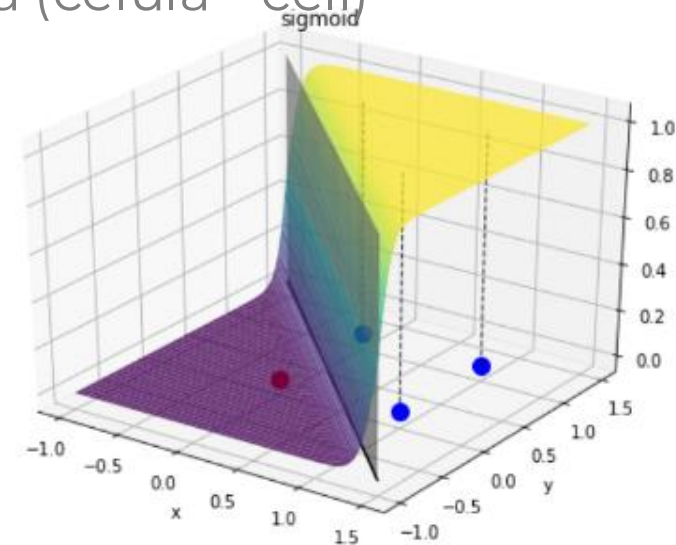


Neurona Biológica

El elemento más pequeño de una ANN es una Neurona (célula - cell)



$$z = \text{sigmoid} \left(\sum_{i=1}^n w_i * x_i + w_0 \right)$$



¿Qué es Perceptrón?

“

Tipo de Red Neural para **clasificación binaria**. Fue inventado en 1950s por Frank Rosenblatt.

”

- Algoritmo de Clasificación.
- Algoritmo Paramétrico
- Entrenamiento Offline.
- Modelo Matemático

NOTA: Existen algoritmos de DL que no son supervisados (i.e.: GAN).



Tipos de Neural Networks

Tipos de Redes Neuronales

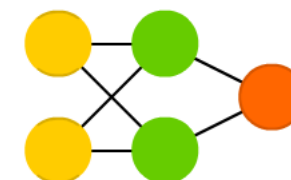
Perceptrón: Es la forma básica de una red neuronal y consta de una capa de entrada, una capa oculta y una capa de salida. Se utiliza principalmente para clasificación binaria.

Perceptron (P)



Redes Neuronales FeedForward (FFNN): Son aquellas en las que la información fluye en una sola dirección, desde la entrada hasta la salida, sin retroalimentación.

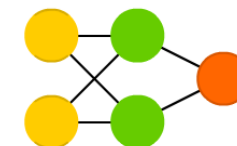
Feed Forward (FF)



Redes Neuronales Radial Basis (RBFF): tipo de red neuronal feedforward que se utiliza para abordar problemas de regresión y clasificación.

- Utilizar una función radial que describe la similitud entre la entrada y un centro o nodo oculto en la red.
- RBF son más sencillas y más rápidas de entrenar.
- Dificultad de manejar problemas con un gran número de clases.
- Difícil selección de los centros para evitar overfitting.

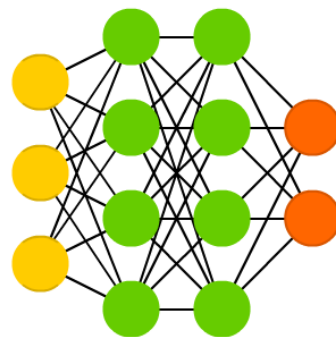
Radial Basis Network (RBF)



Tipos de Redes Neuronales (Profundas)

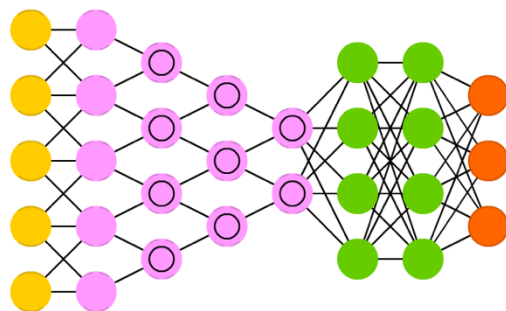
Deep FeedForward (DFFNN): Son FFNN con multiples capas ocultas.

Deep Feed Forward (DFF)

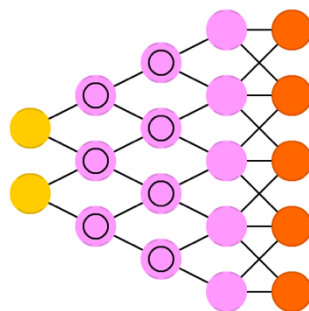


Redes Convolucionales Neuronales (CNNs): Son especialmente útiles para procesar imágenes y se basan en la idea de detectar características locales en una imagen y combinarlas para formar una representación global.

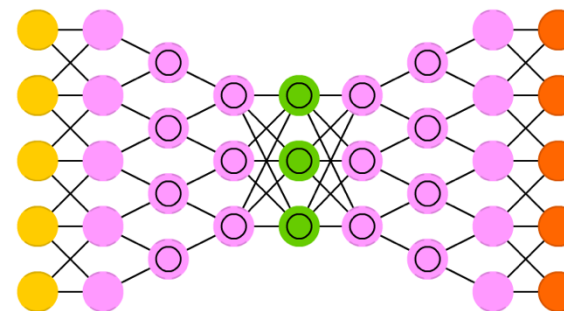
Deep Convolutional Network (DCN)



Deconvolutional Network (DN)



Deep Convolutional Inverse Graphics Network (DCIGN)

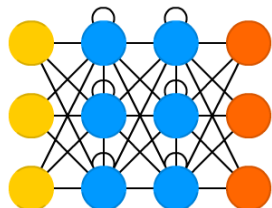


-  Backfed Input Cell
-  Input Cell
-  Noisy Input Cell
-  Hidden Cell
-  Probabilistic Hidden Cell
-  Spiking Hidden Cell
-  Output Cell
-  Match Input Output Cell
-  Recurrent Cell
-  Memory Cell
-  Different Memory Cell
-  Kernel
-  Convolution or Pool

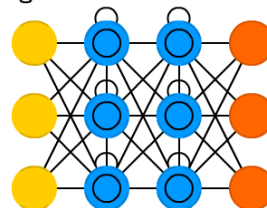
Tipos de Redes Neuronales (Profundas)

Redes Neuronales Recurrentes (RNN): Modelo de redes neuronales que mantienen contexto a lo largo de una secuencia de entradas. Son adecuadas para procesar secuencias de datos como texto, audio o series temporales.

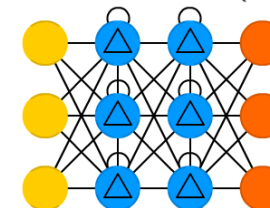
Recurrent Neural Network (RNN)



Long / Short Term Memory (LSTM)

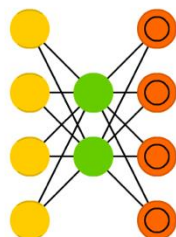


Gated Recurrent Unit (GRU)

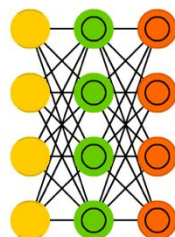


Redes Autoencoders: Son un tipo especial de red neuronal FeedForward en la que la salida es igual a la entrada y se utiliza para aprender una representación compacta de los datos.

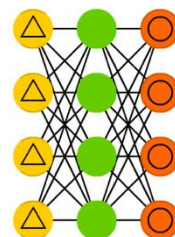
Auto Encoder (AE)



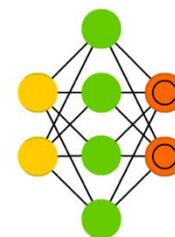
Variational AE (VAE)



Denoising AE (DAE)



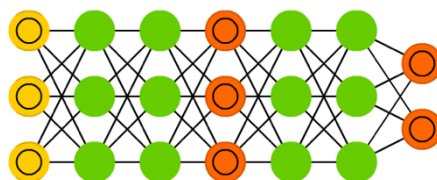
Sparse AE (SAE)



Tipos de Redes Neuronales (Profundas)

Redes Neuronales Generativas Adversarias (GANs): Son una combinación de dos redes neuronales, una que genera datos y otra que los evalúa, que compiten entre sí para producir la mejor representación posible de los datos.

Generative Adversarial Network (GAN)



Redes de atención: Son una forma de procesar secuencias de datos, como texto o audio, donde se pueden poner énfasis en partes específicas de la secuencia en función de un contexto determinado.

Transformers: Son un tipo de modelo de NLP que utilizan una forma de atención para permitir que el modelo considere diferentes pesos para diferentes tokens en la entrada mientras se procesa. (BERT, Codex, GPT-3)

Redes Neuronales Transferenciales: Son un tipo de red neuronal pre-entrenada que se utiliza como punto de partida para abordar tareas específicas, evitando así tener que entrenar una red neuronal desde cero.

Perceptron

Perceptrón

Partes de un Perceptrón:

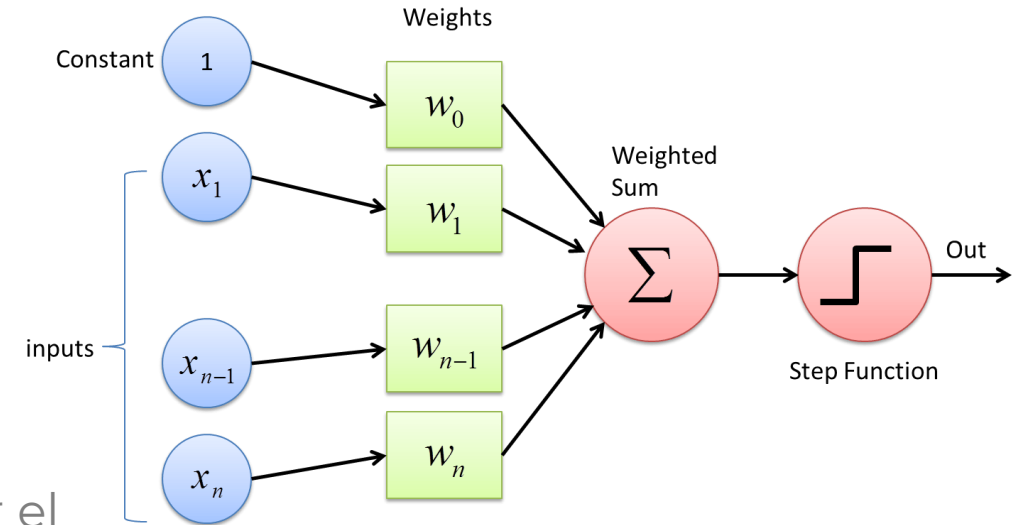
- **Entrada:** numérica de rango [0-1]
- **Pesos w_i :** por cada neurona
- **Bias:** Constante b
- ***Función de activación*** (*Step Function*)
- **Salida (h):** Predicción producida. Toma el valor de 0 o 1 para un problema de clasificación binaria.

Entrenamiento: Ajustar los pesos para minimizar el error en las predicciones.

- **Epocas (Ephocs):** Numero de iteraciones en el proceso de entrenamiento.

Limitaciones:

1. Linealidad
2. Convergencia (linealmente separable)
3. Overfitting



$$h = \text{sigmoid} \left(\sum_{i=1}^n w_i * x_i + w_0 \right)$$

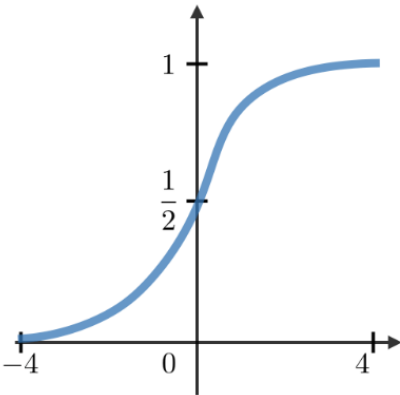
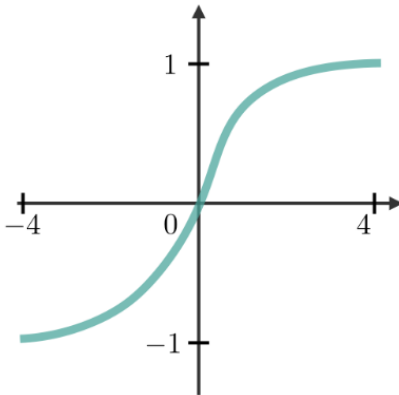
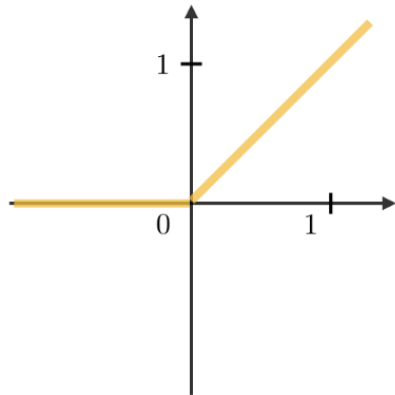
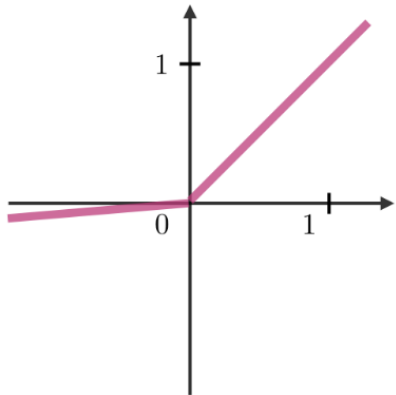
Función de Activación

Las funciones de activación se utilizan al final de una neurona para introducir complejidades **no lineales** en el modelo.

$$\hat{y} = g(w_0 + \mathbf{X}^T \mathbf{W})$$

$$z = \sum_{i=1}^n w_i * x_i + w_0$$

$$\text{where: } \mathbf{X} = \begin{bmatrix} x_1 \\ \vdots \\ x_m \end{bmatrix} \text{ and } \mathbf{W} = \begin{bmatrix} w_1 \\ \vdots \\ w_m \end{bmatrix}$$

Sigmoid	Tanh	ReLU	Leaky ReLU
$g(z) = \frac{1}{1 + e^{-z}}$	$g(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$	$g(z) = \max(0, z)$	$g(z) = \max(\epsilon z, z)$ with $\epsilon \ll 1$
			

$$g'(z) = g(z)(1 - g(z))$$

$$g'(z) = 1 - \tanh^2(z)$$

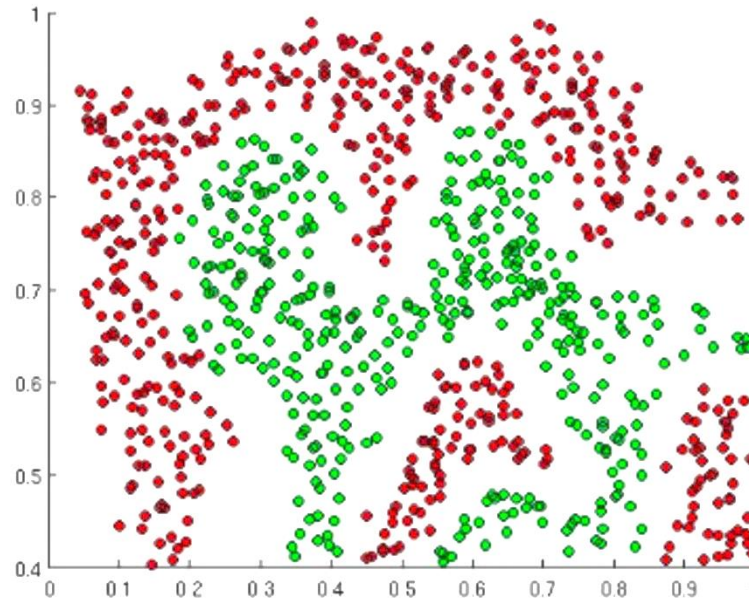
 `tf.math.sigmoid(z)`

 `tf.math.tanh(z)`

 `tf.nn.relu(z)`

Importancia de la Función de Activación

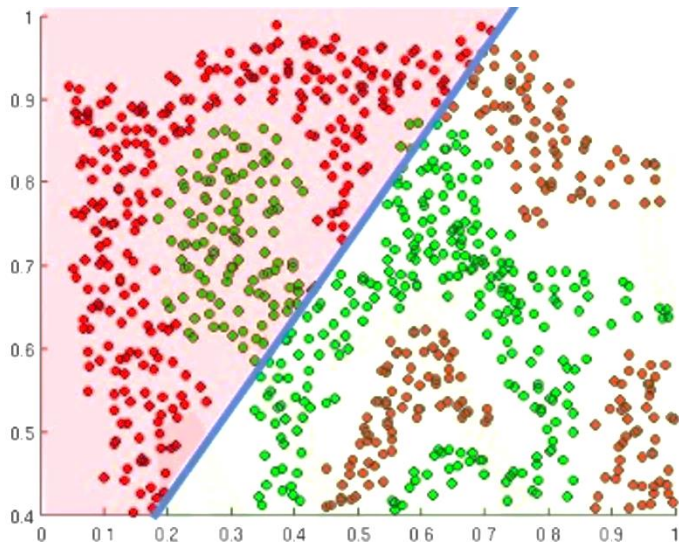
Introducir No-Linealidades dentro de la Red Neuronal



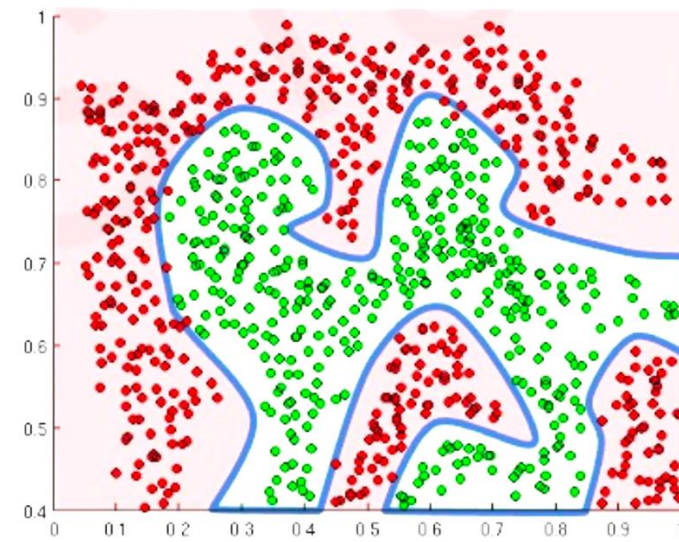
Si quisiéramos construir una Red Neuronal que distinga los punto verdes de los rojos?

Importancia de la Función de Activación

Introducir No-Linealidades dentro de la Red Neuronal



Las funciones de Activación Lineal producen decisiones lineales sin importar el tamaño de la Red Neuronal



No-Linealidades permite aproximar arbitrariamente funciones más complejas

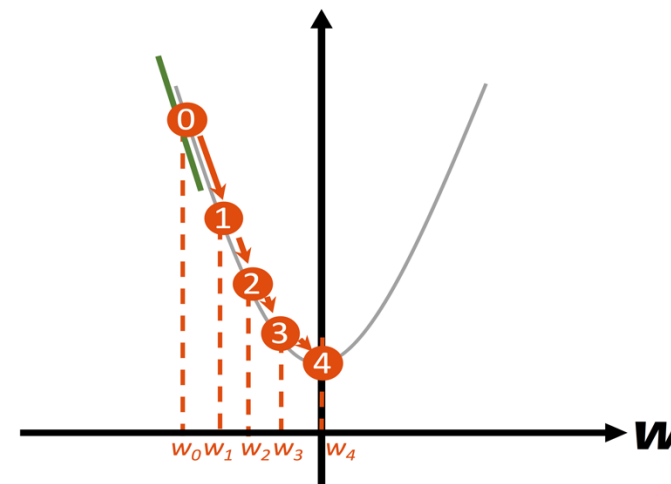
Perceptrón - Entrenamiento

$$w_i \leftarrow w_i + \Delta w_i$$

where

$$\Delta w_i = \eta(t - o)x_i$$

learning rate target value perceptron output input value



x_i : i-esima entrada a la neurona

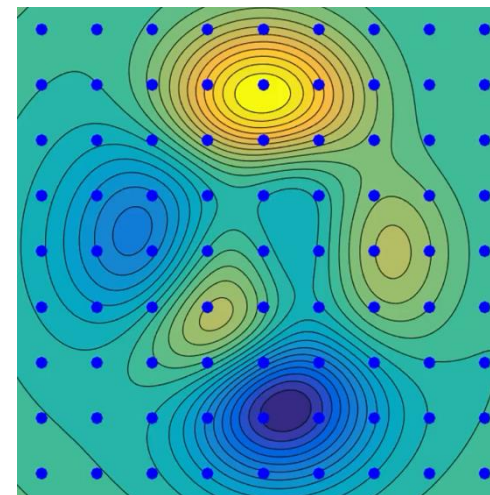
w_i : Pesos de la variable de entrada x_i

η : Ratio de aprendizaje $\mathbb{R}^{0,+}$

t : Valor de salida esperada (valor de la variable clase en x_i)

n : Ratio de aprendizaje $\mathbb{R}^{0,+}$

Nota: Los valores iniciales de w_i son aleatorios.

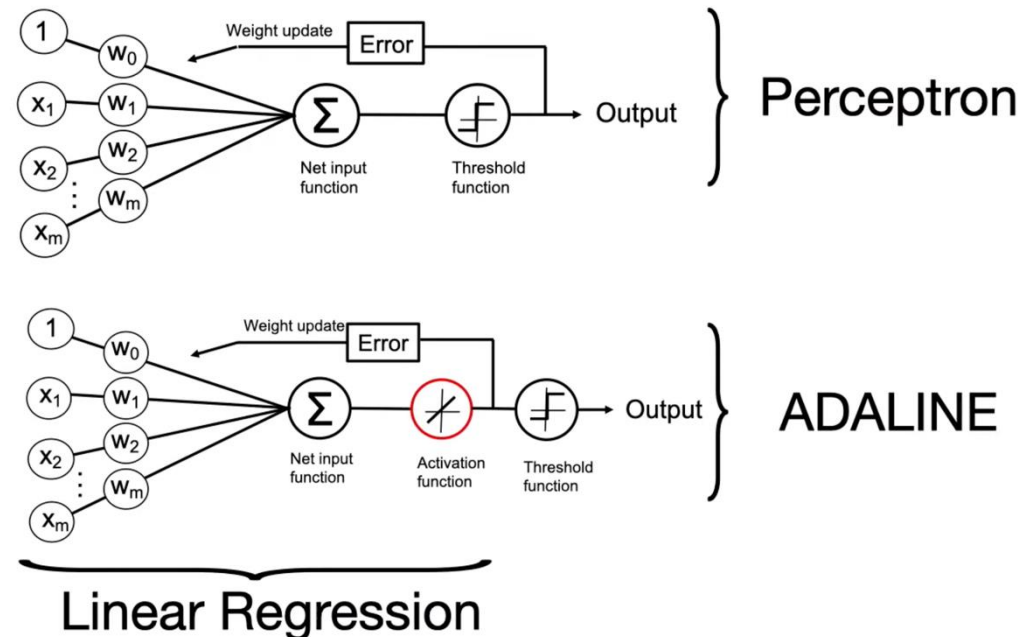


Neurona Adaline (Adaptive Linear Neuron)

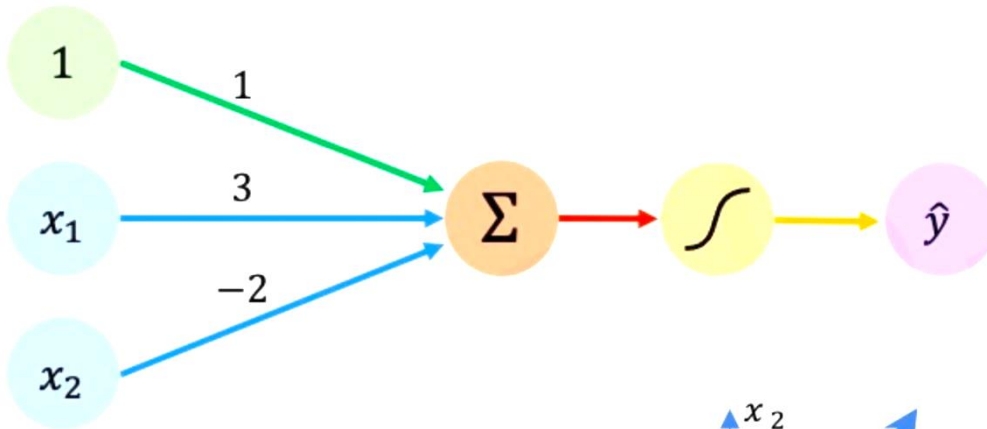
ADALINE es una variante del Perceptron, desarrollada por Bernard Widrow y Ted Hoff en 1960.

- Utiliza una función de activación lineal **Purelin(z)**
- Entrenamiento basado en Gradiente Decendente.

$$h = \text{lin} \left(\sum_{i=1}^n w_i * x_i + w_0 \right)$$

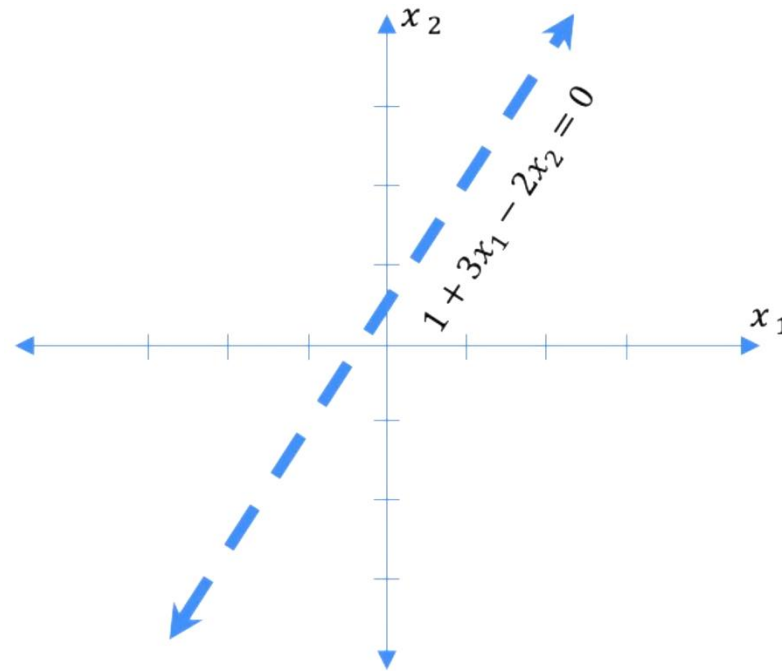


Perceptrón - Ejemplo



$$\begin{aligned}\hat{y} &= g(w_0 + \mathbf{X}^T \mathbf{W}) \\ &= g\left(1 + \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}^T \begin{bmatrix} 3 \\ -2 \end{bmatrix}\right) \\ \hat{y} &= g(1 + 3x_1 - 2x_2)\end{aligned}$$

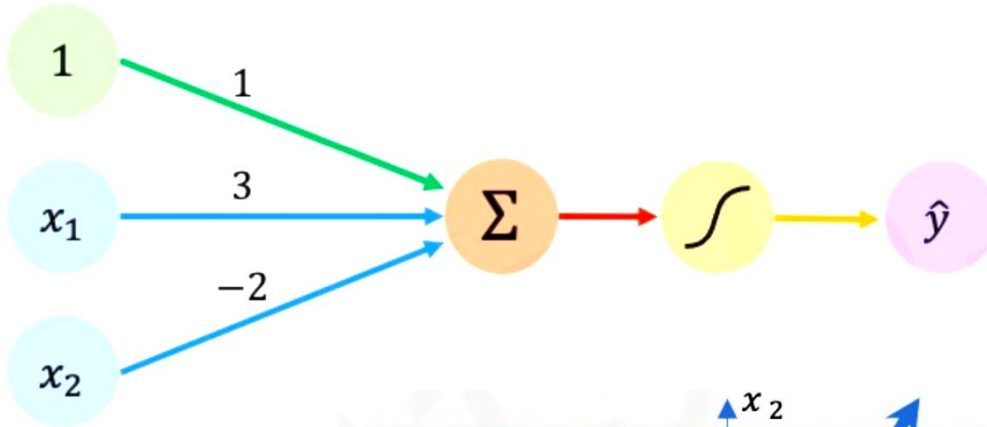
Linea en el Hyperplano 2D



Si:

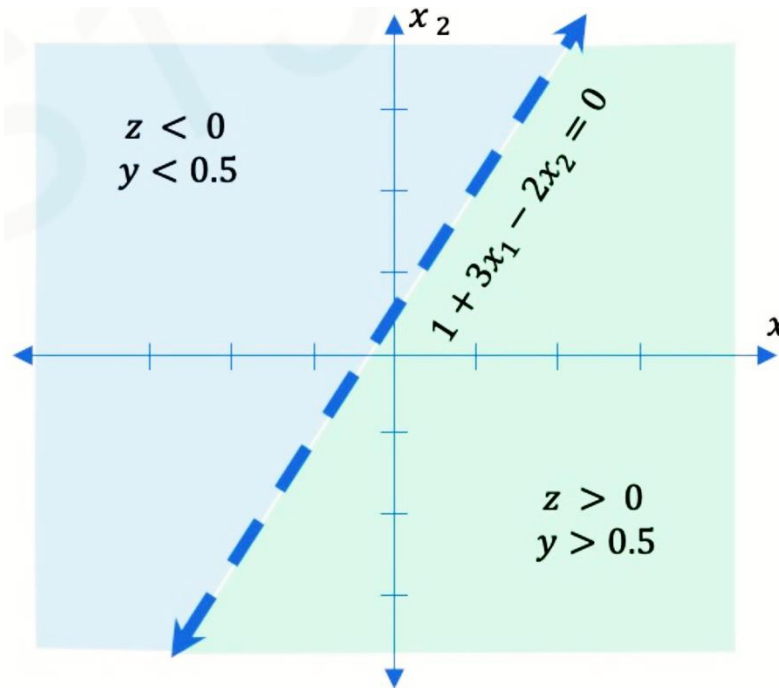
$$\mathbf{X} = \begin{bmatrix} -1 \\ 2 \end{bmatrix}$$

Perceptrón - Ejemplo



$$\begin{aligned}\hat{y} &= g(w_0 + \mathbf{X}^T \mathbf{W}) \\ &= g\left(1 + \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}^T \begin{bmatrix} 3 \\ -2 \end{bmatrix}\right) \\ \hat{y} &= g(1 + 3x_1 - 2x_2)\end{aligned}$$

Linea en el Hyperplano 2D



Si:

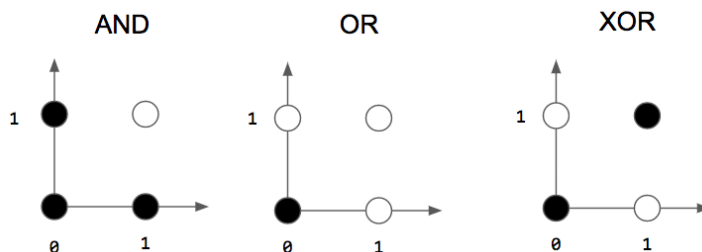
$$\mathbf{X} = \begin{bmatrix} -1 \\ 2 \end{bmatrix}$$

Perceptrón - Python y Numpy

```

1 import numpy as np
2
3 class Perceptron(object):
4     def __init__(self, no_of_inputs):
5         self.weights = np.zeros(no_of_inputs + 1)
6
7     def predict(self, inputs):
8         sum = np.dot(inputs, self.weights[1:]) + self.weights[0]
9         if sum > 0:
10             h = 1
11         else:
12             h = 0
13         return h
14
15     def train(self, training_inputs, labels, epochs=100, learning_rate=0.01):
16         for _ in range(epochs):
17             sum_err = 0.0
18             for inputs, label in zip(training_inputs, labels):
19                 prediction = self.predict(inputs)
20                 self.weights[1:] += learning_rate * (label - prediction) * inputs
21                 self.weights[0] += learning_rate * (label - prediction)
22
23             print("ERROR: ", label - prediction)
24             sum_err += (label - prediction)**2
25         print("SUM ERROR: ", sum_err)
26         if sum_err==0.0:
27             break

```



Dataset Entrenamiento AND

```

[122] 1 X_train = []
      2 X_train.append(np.array([1, 1]))
      3 X_train.append(np.array([1, 0]))
      4 X_train.append(np.array([0, 1]))
      5 X_train.append(np.array([0, 0]))

```

```

[123] 1 y_train = np.array([1, 0, 0, 0])

```

Entrenamiento

```

[124] 1 p = Perceptron(2)
      2 p.train(X_train, y_train, epochs=10)

```

Predicción

```

[125] 1 for x_i in X_train:
      2     print(x_i, p.predict(x_i))

```

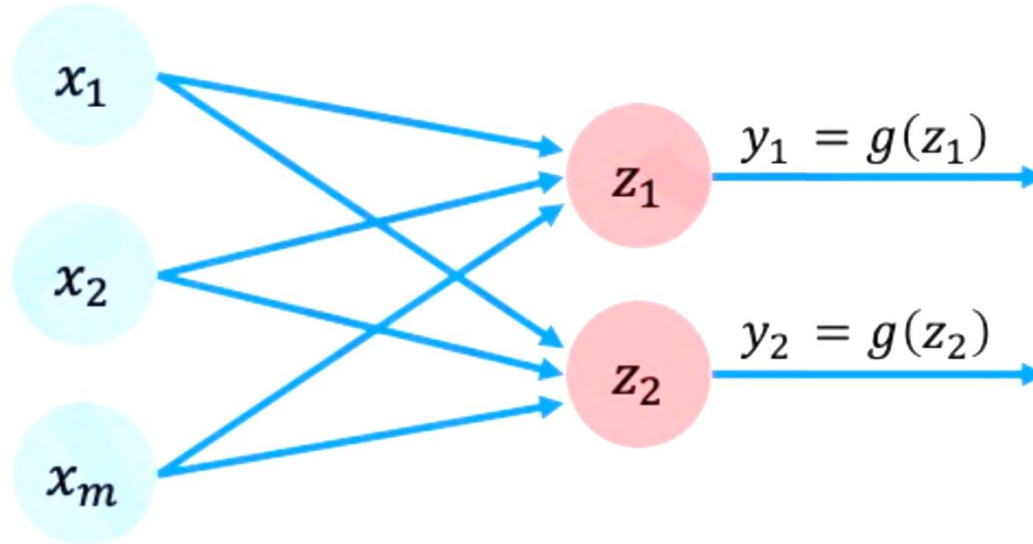
```

[1 1] 1
[1 0] 0
[0 1] 0
[0 0] 0

```


Perceptrón con Múltiples Salidas

Porque las entradas están densamente conectadas (todos con todos), a estas capas se llaman **Dense Layers**.



```
import tensorflow as tf  
  
layer = tf.keras.layers.Dense(  
    units=2)
```

$$z_i = w_{0,i} + \sum_{j=1}^m x_j w_{j,i}$$



Feed Forward NN (FFNN)

Feed Forward Neural Network (FFNN)

Multi-Layer Perceptron (MLP):

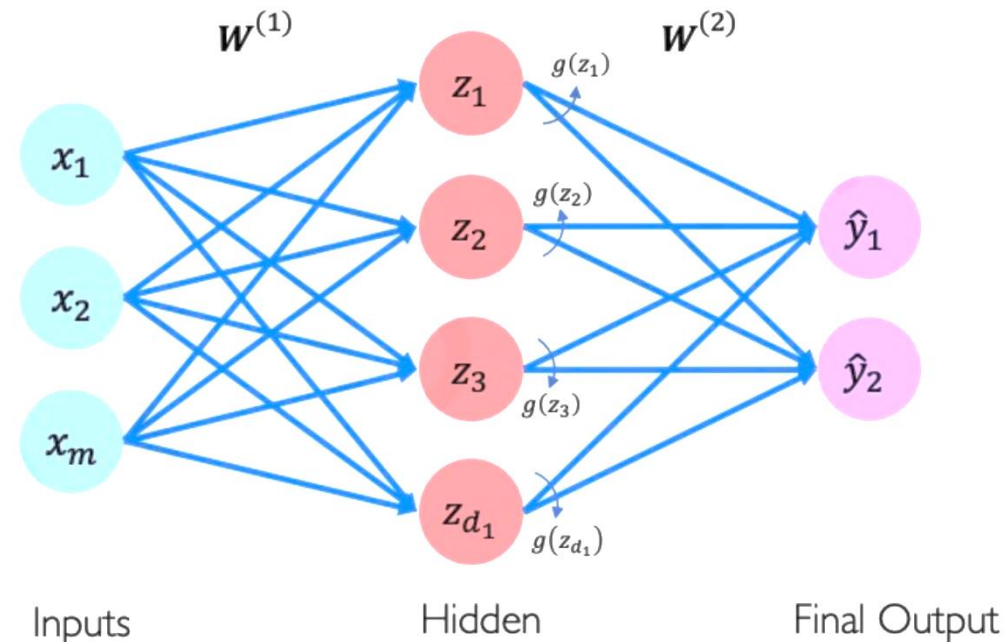
- **Entrada:** numérica de rango [0-1]
- **Pesos w_{ji} :** por cada peso j en la i neurona
- w_{j0} : Constante
- **Función de activación** (*Step Function*)
- **Salida (z_{0i}):** Predicción producida por cada neurona.

Entrenamiento: Ajustar los pesos para minimizar el error cuadrático con algoritmo **Backpropagation**.

- **Epocas (Epochs):** Numero de iteraciones en el proceso de entrenamiento.

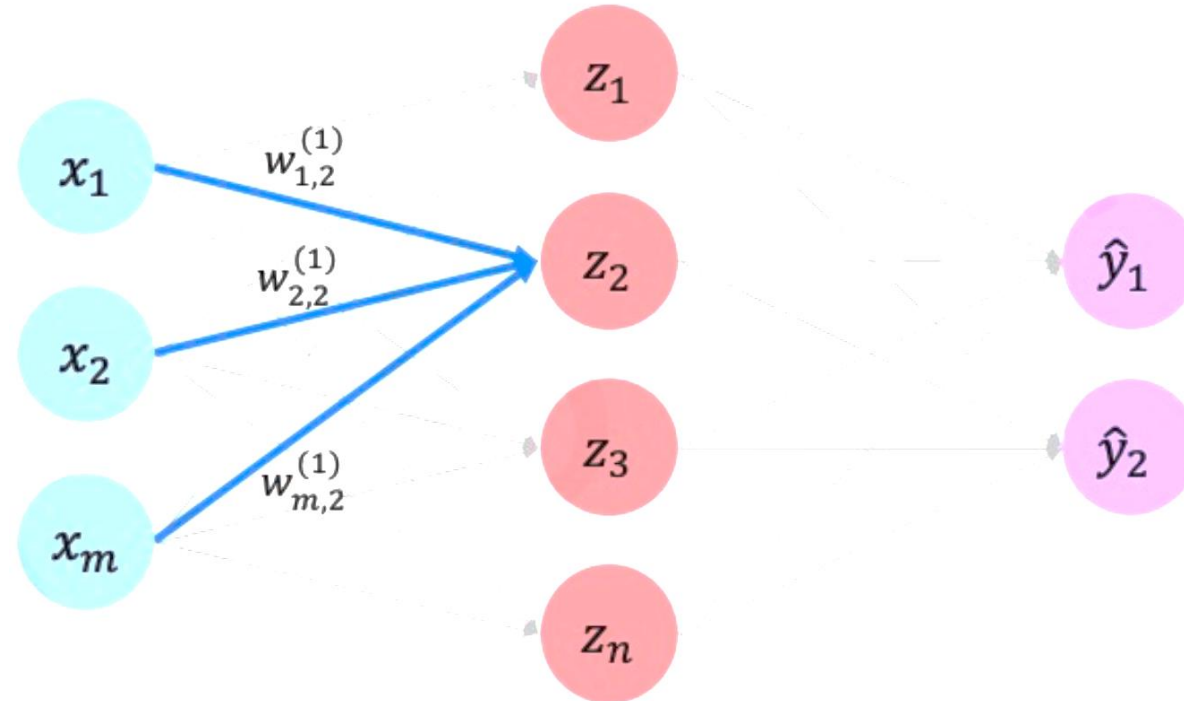
Limitaciones:

1. Underfitting
2. Overfitting



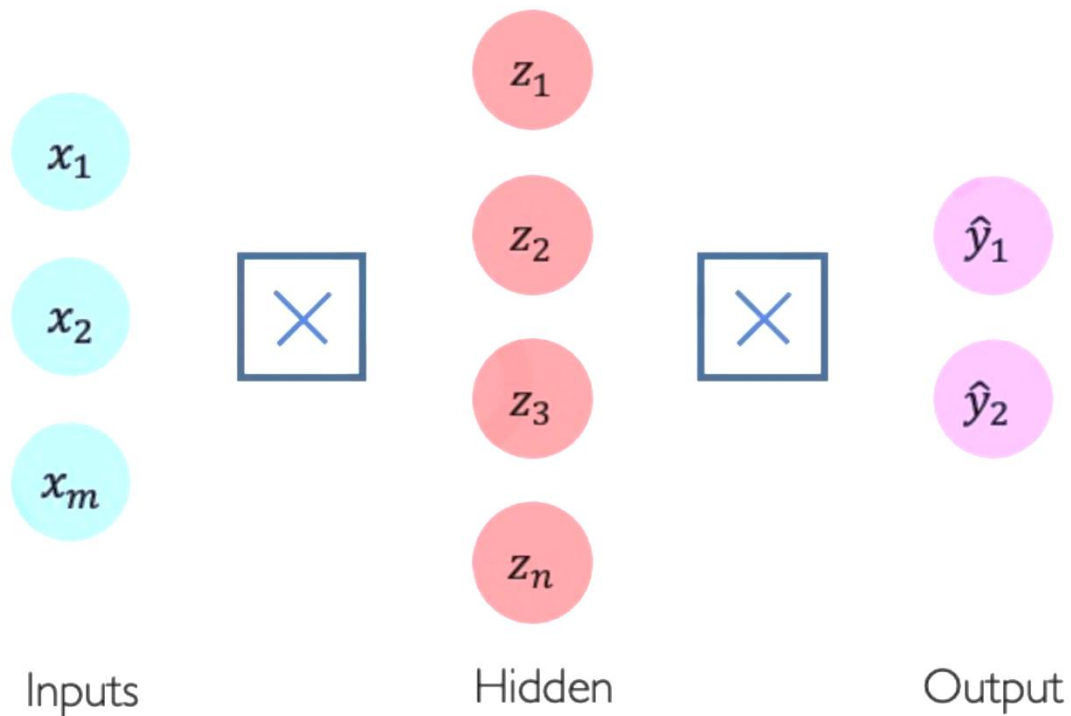
$$z_i = w_{0,i}^{(1)} + \sum_{j=1}^m x_j w_{j,i}^{(1)} \quad \hat{y}_i = g \left(w_{0,i}^{(2)} + \sum_{j=1}^{d_1} g(z_j) w_{j,i}^{(2)} \right)$$

Feed Forward Neural Network (FFNN)



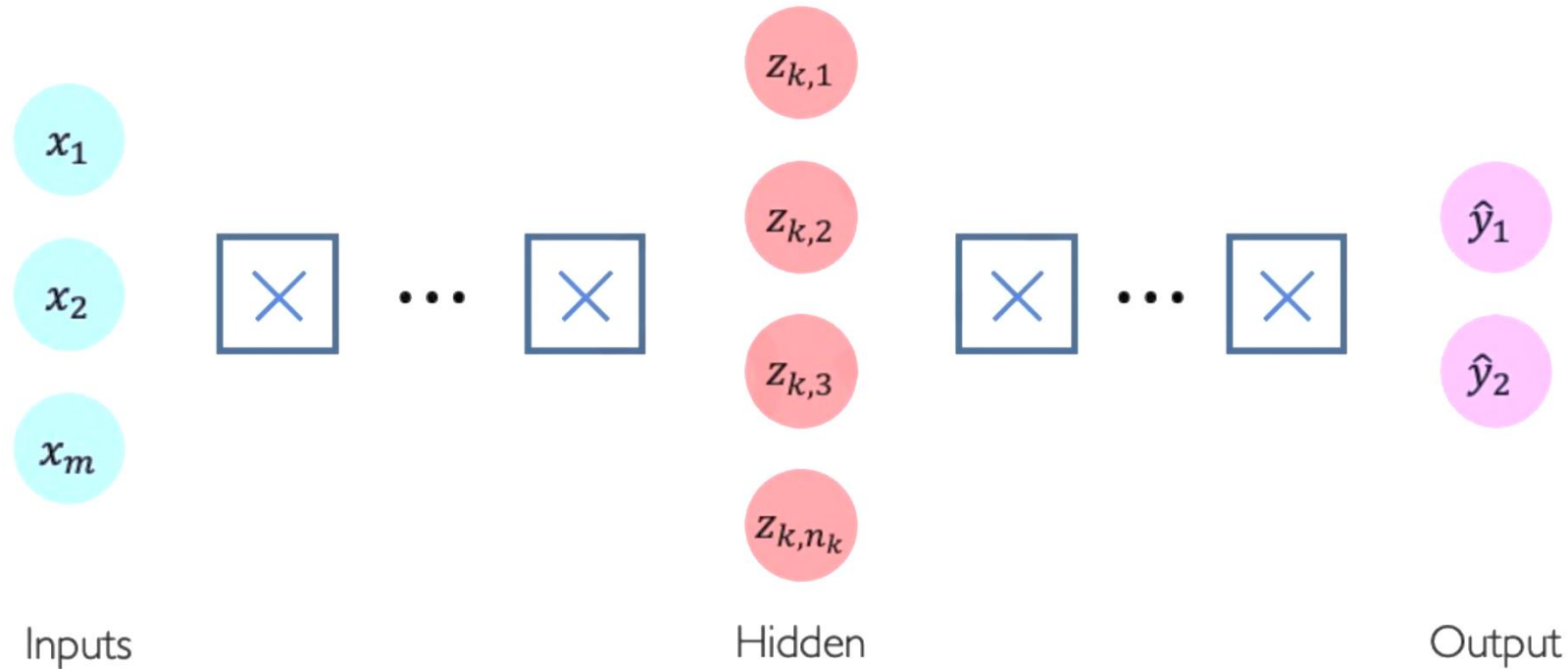
$$\begin{aligned} z_2 &= w_{0,2}^{(1)} + \sum_{j=1}^m x_j w_{j,2}^{(1)} \\ &= w_{0,2}^{(1)} + x_1 w_{1,2}^{(1)} + x_2 w_{2,2}^{(1)} + x_m w_{m,2}^{(1)} \end{aligned}$$

Feed Forward Neural Network (FFNN)



```
 import tensorflow as tf  
  
model = tf.keras.Sequential([  
    tf.keras.layers.Dense(n),  
    tf.keras.layers.Dense(2)  
])
```

Deep Feed Forward Neural Network (DFFNN)

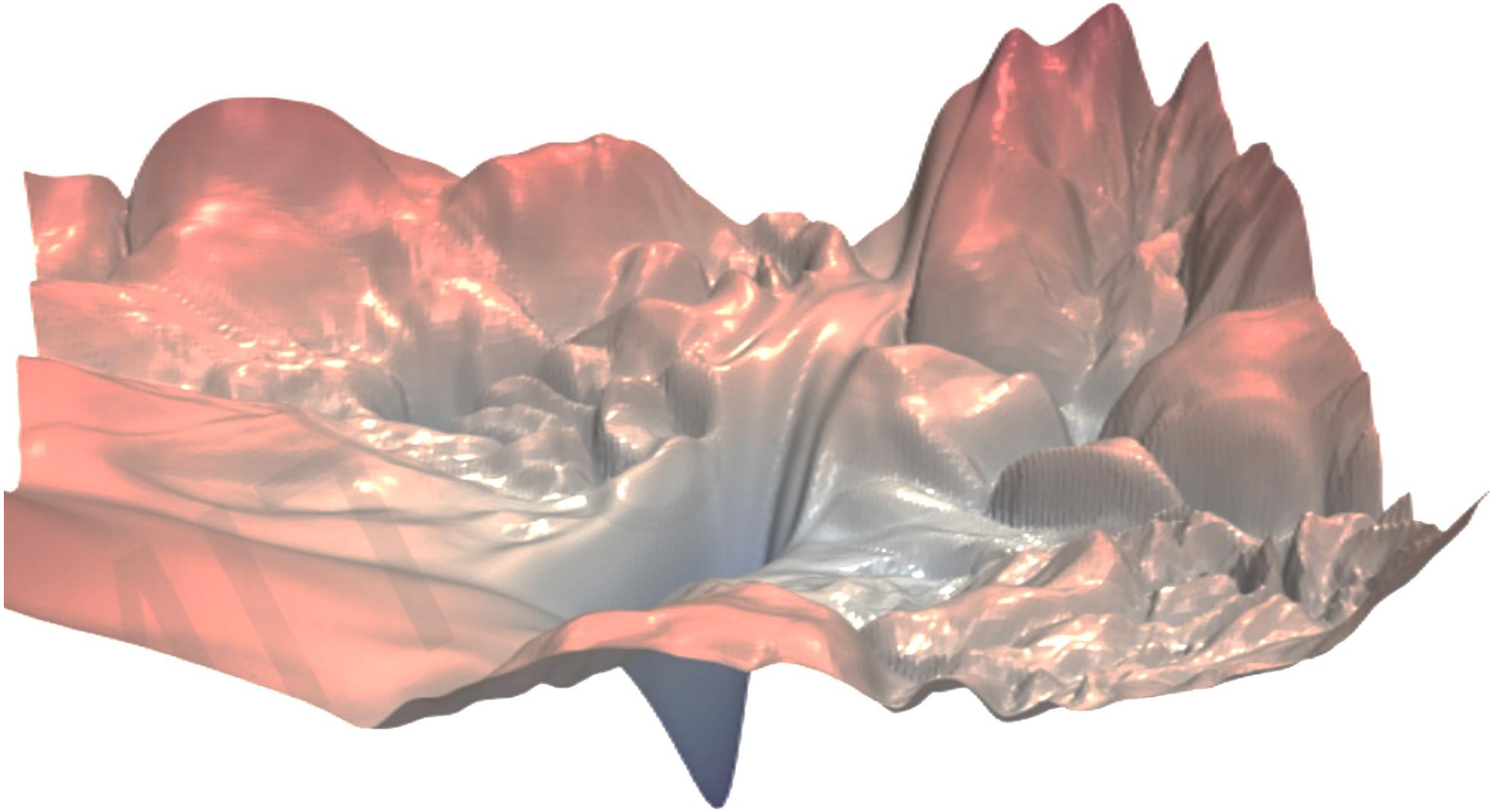


```
import tensorflow as tf

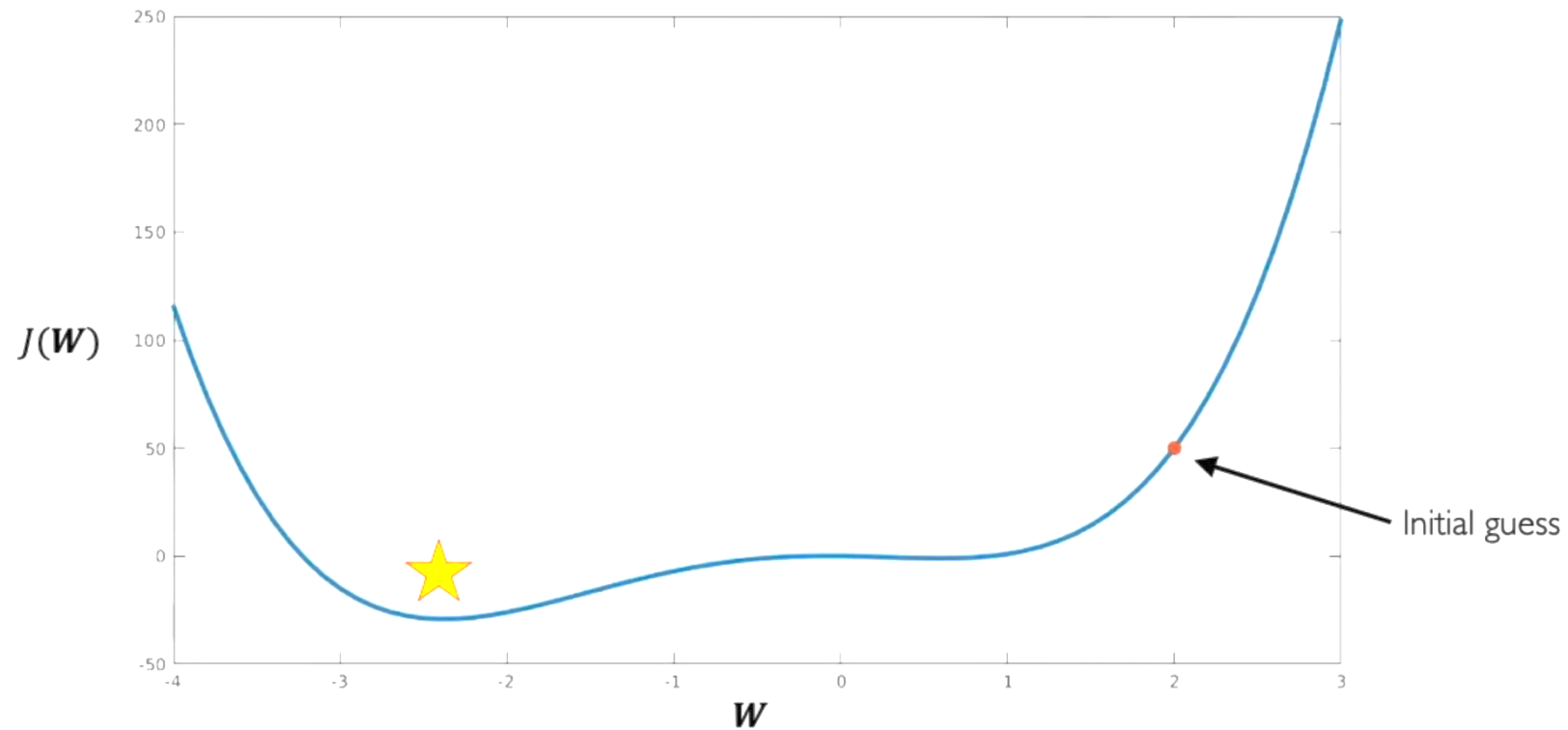
model = tf.keras.Sequential([
    tf.keras.layers.Dense(n1),
    tf.keras.layers.Dense(n2),
    :
    tf.keras.layers.Dense(2)
])
```

$$z_{k,i} = w_{0,i}^{(k)} + \sum_{j=1}^{n_{k-1}} g(z_{k-1,j}) w_{j,i}^{(k)}$$

Entrenamiento de NN es difícil !!!



La Función de Costo es difícil de Optimizar



$$W \leftarrow W - \eta \frac{\partial J(W)}{\partial W}$$

How can we set the learning rate?

La Función de Costo es difícil de Optimizar



Para lidiar con el “*learning_rate*” se tienen 2 estrategias:

IDEA 1:

Intentar el entrenamiento con diferentes *learning_rate* hasta que se encuentre convergencia.

IDEA 2:

Diseñar una estrategia adaptativa de aprendizaje que se adapte a la curva de costo

Algoritmos del Gradiente Descendente

Algorithm

- SGD
- Adam
- Adadelta
- Adagrad
- RMSProp

TF Implementation



```
tf.keras.optimizers.SGD
```



```
tf.keras.optimizers.Adam
```



```
tf.keras.optimizers.Adadelta
```



```
tf.keras.optimizers.Adagrad
```



```
tf.keras.optimizers.RMSProp
```

Reference

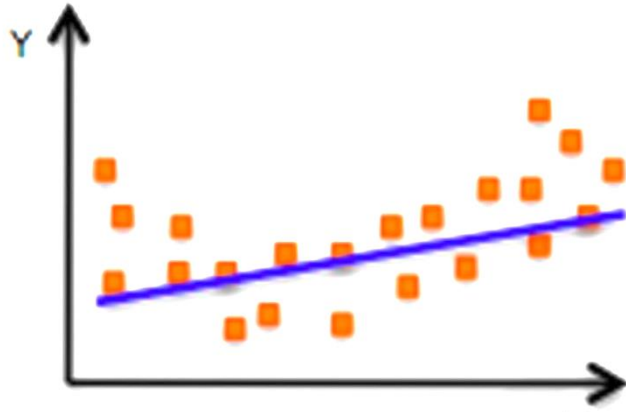
Kiefer & Wolfowitz. "Stochastic Estimation of the Maximum of a Regression Function." 1952.

Kingma et al. "Adam: A Method for Stochastic Optimization." 2014.

Zeiler et al. "ADADELTA: An Adaptive Learning Rate Method." 2012.

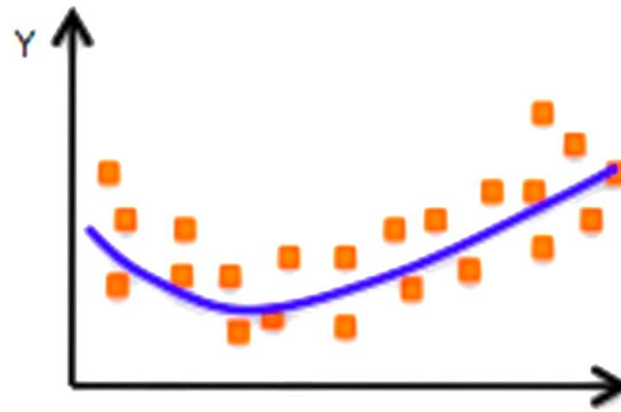
Duchi et al. "Adaptive Subgradient Methods for Online Learning and Stochastic Optimization." 2011.

Problemas de Overfitting

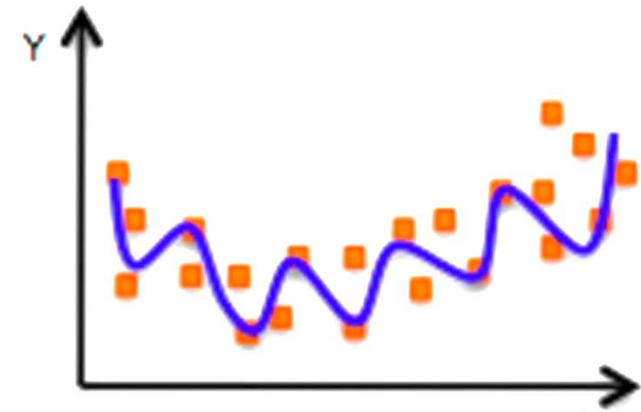


Underfitting

El modelo no tiene la capacidad de aprender acerca de los datos.



IDEAL



Overfitting

Modelo complejo, con parámetros extra y no generaliza el comportamiento de los patrones

Regularización

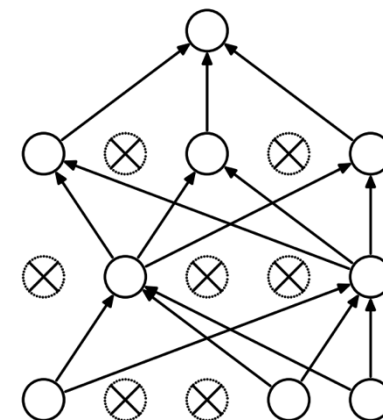
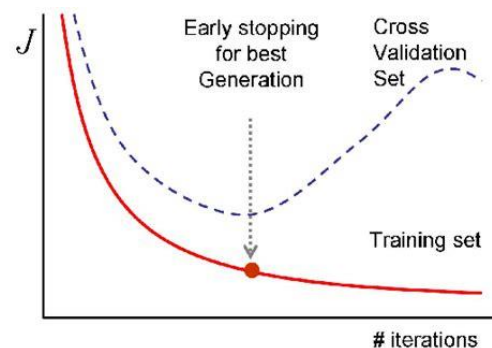
Definición:

Son técnicas que restringen el modelo de optimización para simplificar el modelo.

Resultado: Mejora la generalización del modelo hacia datos que no se utilizan en el entrenamiento.

Tipos de Regularización en NN:

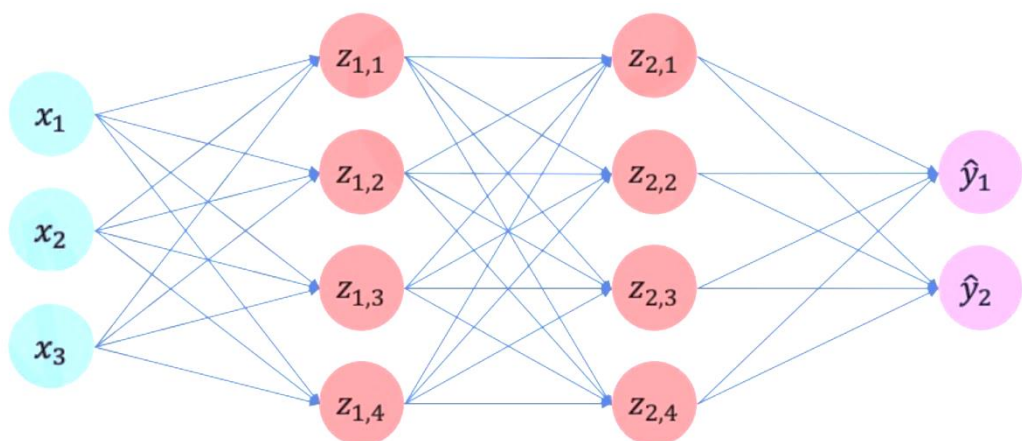
- Regularización por Dropouts
- Detención temprana (Early Stop)



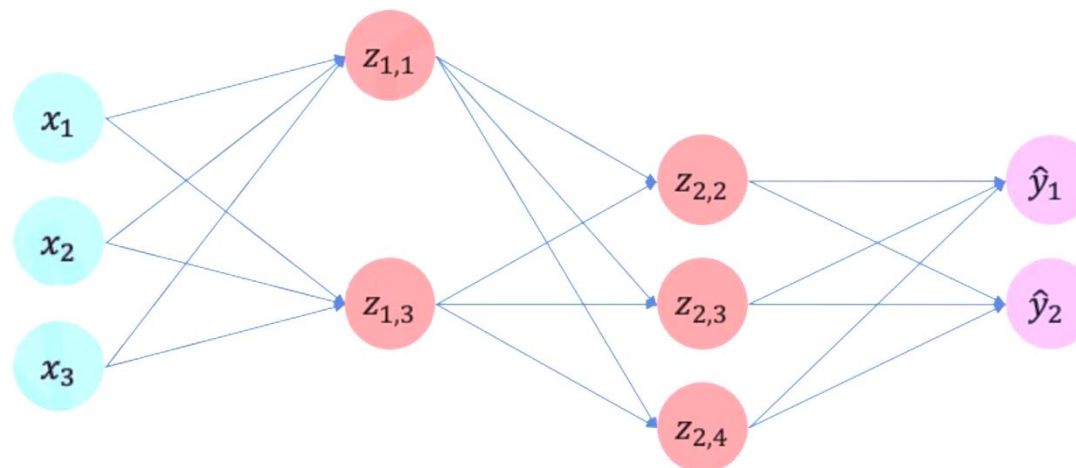
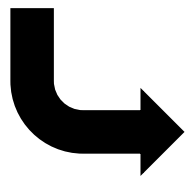
Regularización - Dropouts

Durante el entrenamiento, aleatoriamente las activaciones se vuelven 0.

- Típicamente se usa **un “drop” de 50%** en cada capa.
- Obliga a la NN a no usar la neurona “i” en una capa dada.



 `tf.keras.layers.Dropout(p=0.5)`



Regularización – Early Stop

Se detiene el proceso de entrenamiento cuando existe indicio de Overfitting.

- El Error con los datos de testeo empieza a subir.

